

MicroGreen – DSL de Estufa/Aquecimento sobre MicrowaveVM

APS – LogComp

Gabriel Pessine Buhrer

Professor: Raul Ikeda

2025.2

Motivação

- Operador de estufa / aquecedor normalmente **não é programador**.
- Linguagens tradicionais (C, Python, etc.) são **complexas** para esse usuário.
- Ideia: criar uma DSL com comandos diretos:
 - `potencia 80`
 - `aquecer ate 60`
 - `manter acima de 55 por 20s`
- O compilador traduz essas frases para um **assembly simples**, executado por uma VM (MicrowaveVM).

Características da MicroGreen

- Linguagem de **altíssimo nível** focada em **estufas/aquecimento**.
- Sintaxe em **português**, com **uma instrução por linha**.
- Comandos principais:
 - potencia N
 - esperar Ts
 - aquecer ate X
 - manter acima de X por Ts
 - alarme se temperatura REL K
 - mostrar temperatura / mostrar potencia
 - bipe / parar

Características da MicroGreen

- Estruturas de programação (variáveis/condicionais/loops) ficam **escondidas**:
 - loops implementados com TIME + DECJZ + GOTO
 - “estados” guardados em registradores e pilha

Gramática (EBNF)

- A linguagem foi especificada em EBNF e depois implementada com Flex (léxico) e Bison (sintaxe + geração de código).

```
PROGRAMA = { LINHA } ;
LINHA = [ COMANDO ], ( "\n" | ";" ) ;

COMANDO = AQUECER | MANTER | VEL | ALARME | ESPERAR | MOSTRAR | BIPE | PARAR ;

AQUECER = "aquecer", "ate", INT ;
MANTER = "manter", "acima", "de", INT, "por", DUR ;
VEL = "potencia", INT ;
ALARME = "alarme", "se", "temperatura", REL, INT ;
REL = "maior" | "menor" | "igual" ;
ESPERAR = "esperar", DUR ;
MOSTRAR = "mostrar", ( "temperatura" | "potencia" ) ;
BIPE = "bipe" ;
PARAR = "parar" | "halt" ;

COMENT = "#", { CAR }, ( "\n" | EOF ) ;

INT = DIGITO, { DIGITO } ;
DUR = INT, "s" ;
CAR = "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H" | "I" | "J" | "K" | "L" | "M" | "N" | "O" | "P" | "Q" | "R" | "S" | "T" | "U" | "V" | "W" | "X" | "Y" | "Z"
      | "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h" | "i" | "j" | "k" | "l" | "m" | "n" | "o" | "p" | "q" | "r" | "s" | "t" | "u" | "v" | "w" | "x" | "y" | "z" ;
DIGITO = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ;
```

Arquitetura do Compilador

- Lexer (microgreen.l): transforma texto em tokens (T_POTENCIA, T_INT, T_DUR, etc.).
- Parser (microgreen.y): reconhece a gramática e, em cada comando, executa `fprintf(mg_out, "...")`.
- main.c: integra tudo, abre o .mgreen, chama `yyparse()` e escreve o .mwasm.
- python3 main.py programa.mwasm (MicrowaveVM) executa o assembly.

Arquitetura do Compilador



Exemplo 1

Código MicroGreen:

```
potencia 80
aquecer ate 60
parar
```

Assembly .mwasm gerado:

```
; potencia 80
SET POWER 80

; aquecer ate 60 (modelo
simplificado)
SET TIME 60
L_aquecer_0:
DECJZ TIME L_aquecer_fim_0
GOTO L_aquecer_0
L_aquecer_fim_0:

; parar
HALT
```

Execução na VM (terminal):

Loaded program from:
01-aquecer.mwasm

BEEEEEEP!

Final state: {'TIME': 0,
'POWER': 80}

Final readonly state:
{'TEMP': 345,
'WEIGHT': 100}

Final stack: []

Exemplo 2

Código MicroGreen:

potencia 70
manter acima de 55
por 20s
parar

Assembly .mwa sm gerado:

```
; potencia 70
SET POWER 70
; manter acima de 55 por 20
segundos (modelo
simplificado)
SET TIME 20
L_manter_0:
DECJZ TIME L_manter_fim_0
GOTO L_manter_0
L_manter_fim_0:
; parar
HALT
```

Execução na VM (terminal):

Loaded program from:
../microgreen/02-
manter.mwasm
BEEEEEEP!

Final state: {'TIME': 0,
'POWER': 70}

Final readonly state:
{'TEMP': 189,
'WEIGHT': 100}

Final stack: []

Exemplo 3

Código MicroGreen:

```
potencia 40
alarme se
  temperatura maior
  70
mostrar temperatura
esperar 10s
parar
```

Assembly .mwa sm gerado:

```
; potencia 40
SET POWER 40

; alarme se temperatura ... 70 (modelo simplificado;
  sempre dispara)

PUSH TIME
PRINT
POP TIME

; mostrar temperatura (aprox: imprime TIME)
PRINT

; esperar 10 segundos
SET TIME 10

L_wait_0:
DECJZ TIME L_end_0
GOTO L_wait_0

L_end_0:

; parar
HALT
```

Execução na VM (terminal):

Loaded program from:
../microgreen/03-
alarme.mwasm

TIME: 0

TIME: 0

BEEEEEEEP!

Final state: {'TIME': 0,
'POWER': 40}

Final readonly state:
{'TEMP': 72, 'WEIGHT':
100}

Final stack: []

Curiosidades e Decisões de Design

- **Sintaxe em português** (“potencia”, “aquecer ate”, “manter acima de”, “maior/menor/igual”) para ficar o mais natural possível.
- Linguagem **não expõe** `if`, `while`, variáveis ou registradores.
 - O compilador gera `SET`, `DECJZ`, `GOTO`, `PUSH`, `POP`, `PRINT` e `HALT` internamente.
- A VM não permite `PUSH TEMP`:
 - Por isso, mostrar temperatura imprime **TIME como aproximação** (proxy de aquecimento).
- `alarme se temperatura ... está em uma` **semântica simplificada**:
 - sempre dispara o beep em vez de realmente checar `TEMP` na VM.

Possíveis Evoluções

- melhorar a modelagem da relação tempo/temperatura,
- implementar checagem real de TEMP (se a VM suportar),
- adicionar comandos de “estágios” (stage, “perfil de aquecimento”, etc.).

Mais informações

- O arquivo README.md do projeto contém mais detalhes e está localizado na pasta raiz do repositório.

https://github.com/GabrielBuhrer/LogComp_APS